

Analysis of sun.f

A FORTRAN 77 model of sunspot umbra/penumbra evolution — prepared ahead of a Python port

Source file: sun.f **Prepared:** August 27, 2026

Big picture

This is a 1990s/2000s-era FORTRAN 77 program that models how sunspot **umbra** (dark core) and **penumbra** (surrounding lighter region) evolve over time, based on a paper by Bogdan et al. (Science, 6 Feb 2004) on sunspot pore structure. It:

- Reads pre-processed sunspot observation data (`rgo_data.prn`, from the Royal Greenwich Observatory) — daily counts of sunspot area (CWSA = corrected whole spot area, CUA = corrected umbra area) per group.
- Asks the user interactively for calibration parameters (Umbra count, total sunspot count, and three percentages).
- Runs a batch loop over “Carrington rotations” (27-day solar rotation periods), computing a predicted umbra/pore evolution value for each day using an empirical exponential-decay model (EXPO and its helpers).
- Separately, an epidemic-style differential-equation model (SPUR, borrowed from disease-spread modeling — S-E-I-R: Susceptible/Exposed/Infected/Removed) simulates how “infection” (spot state) spreads between 8 polygon cells on the solar disk, using a simple numerical relaxation method.
- Writes results to `SUN.CSV`.

It is really two loosely-coupled models glued together: an empirical curve-fit (EXPO/ENERGY/MASS/JTEMP/E2) driving the main per-day output, and an SIR-style compartmental model (SPUR/ZCELLS/MOVES) meant to represent spatial spreading between regions, whose output barely feeds back into the main calculation (it's summed into POSQ only for the first record of each batch, then unused).

PROGRAM SUN — main program (lines 1–206)

Setup

`IMPLICIT DOUBLE PRECISION` means any variable starting with A–H or O–Z defaults to double precision — classic FORTRAN convention, and the reason the code has almost no explicit type declarations. The program opens `rgo_data.prn` for input and `SUN.CSV` for output.

User prompts

Five values are read interactively and echoed into the CSV header:

Variable	Prompt	Notes
----------	--------	-------

beta	Number of Umbra seen	used as a scale constant $C = \text{beta} * \text{ICUA}$
rr	Number of total sunspots seen	converted to a fraction: $rr = (rr+1)/100$
delta	% of Umbra area with Penumbra	used as a filter/scale multiplier
g	% of sunspots in Umbra	used with alpha to scale SUSQI
P	% of Umbra area to total group area	$Pr = (1-P)/10$, $x = 1/Pr$

$K=1$ and $\alpha=.95$ are hardcoded — these were originally user prompts too, now commented out, suggesting someone simplified the interface at some point.

Main DO WHILE loop

Each pass through the outer loop asks the user “Continue (1), Stop (0)” — this is a manual, semi-interactive batch process rather than a single run-to-completion program. While continuing, it loops `DO 100 IREC = NREC, NBATCH`, reading one record per day from `rgo_data.prn`: group id (ICSG), day, whole-spot area (ICWSA), and umbra area (ICUA), among others.

- For each record, derives model parameters (RINF, freq, C, PG) as simple multiples of ICUA, scaled by the user's calibration percentages.
- **First K days of a batch** ($K=1$, so effectively just the first record): calls ZCELLS (the SPUR/epidemic submodel) to get an INF array, sums it into F, then calls EXPO to compute an evolution estimate from the empirical formulas.
- **Remaining days**: uses a simple decay $\text{POSQ} = 1/\text{IREC}$ instead of the empirical model.
- Also computes a REMQ “removal” term, but only on odd IREC values.
- Combines everything into SUSQS (sunspot-count estimate) and SUSQI (umbra/pore estimate) via formulas resembling discrete diffusion/logistic terms, each rescaled again by $\text{delta} / g / \alpha$.
- Writes one CSV row per record: day, SUSQI, SUSQS, ICUA, ICWSA.
- Accumulates avepores (running sum of SUSQS) — reported back to the user as npores / nspots at the top of the next outer-loop pass.
- After a batch, advances NREC/NBATCH to the next chunk; if reading from the RGO data file, N is re-derived from the last day seen as roughly 3 Carrington rotations ($3 \times \text{days} - 3$).

Error handling

Labels 350 / 400 / 60 are classic GOTO-based error handlers for file read/write failures, reporting where execution broke via LOC (a manually maintained “line marker” variable) and IREC.

SUBROUTINE EXPO and helpers — the empirical curve-fit model

Given time, PG, RINF, freq, and C, EXPO computes an output F. This is explicitly an ad-hoc curve fit (“SICE”, roughly a saturation/inflection curve estimate) built from four sub-formulas — ENERGY (H), MASS (M), JTEMP (J), and E2 (ESQR) — that all follow the same pattern: take $\ln(\text{time})$, turn it into an angle via arctan, then exponentiate back out. The header comment block documents the intended formulas, though

variable names in code don't always match the comments one-to-one (e.g. `v` in the comments corresponds to `time`).

Subroutine	Output	Shape
ENERGY	H	$\exp(-\text{atan}(\tan(\text{acos}(1/(\ln(\text{time}) \cdot \pi)))))$ -style sigmoid of $\ln(\text{time})$
MASS	M	same transform shape, scaled by .4 instead of π ;
JTEMP	J	nested log/exp/atan transform, guarded for $\ln(\text{time}) \leq 0$
E2	ESQR	similar transform combined with π ;

These feed into the final combination:

```
K = J3    KT = log10(K·PG) (floored at 2.5 if ≤ 0)
SICE = [1 / (3π·H3·freq3)] · [3·√ESQR / (4π·M·freq)]KT
F = SICE · freq · M · C
```

This is essentially a hand-fit empirical function with no obvious physical derivation — it approximates an S-shaped (saturating) growth curve. CPI (computing $c = \exp(\exp(10/\pi))$, a very large constant) is dead code — it's never called from anywhere in the program.

ZCELLS / SPUR / MOVES / ran2 — the epidemic (SEIR) submodel

This is a separate epidemiological SIR/SEIR model — literally borrowed (the header comment says “simulate Olsen and Schaffer's SPUR equations”, and variable names S/E/I/R map to Susceptible/Exposed/Infected/Removed) and repurposed to model 8 polygon “cells” on the solar disk, presumably one per active region or quadrant.

ZCELLS(timestep, inf)

- If `timestep == 0`: reads initial conditions from `polypara.dat` (8 rows: polygon ID, S/E/I/R fractions, a/g/m rate constants, `B` = beta/contact-rate seed), calls SPUR for each polygon, and writes results to `TIMENEW.DAT`.
- Otherwise: reads previous state from `TIMEOLD.DAT`, applies MOVES to jitter the contact rate `B` via a random-number draw, re-runs SPUR, and rotates `TIMENEW.DAT` → `TIMEOLD.DAT` for the next call.
- Returns `inf(8)` — the “Infected” fraction per polygon after settling — interpreted by the main program as pore-visibility percentages.

SPUR

A classic explicit numerical relaxation of the SEIR ODEs (dS/dt , dE/dt , dI/dt , dR/dt), using a birth/death rate `1/LIFEXPT` and a fixed-point “average-and-relax” inner loop, run up to 50 outer iterations, each inner-looping until percent changes converge under `CRIT = .00001`. The final `IAV(10)` (infected fraction at the 10th of 11 sampled points) becomes the polygon's output.

MOVES / ran2

`ran2` is the classic *Numerical Recipes* linear-congruential random-number generator with a shuffle table. MOVES draws one such value to perturb `B` (the transmission/contact-rate parameter) between timesteps,

simulating some randomness in sunspot region movement.

Important wiring note: in the main program, K=1 means the ZCELLS/EXPO combined branch only executes for the very first IREC of each batch (IREC.LE.K), and ZCELLS is always called with timestep = IREC-1, i.e. typically timestep = 0 — so it mostly just re-reads polypara.dat fresh every time rather than evolving TIMENEW.DAT/TIMEOLD.DAT across calls. This SEIR submodel is largely vestigial/underused as currently wired into the main loop.

Data files

File	Role
rgo_data.prn	Input: daily sunspot group observations (group id, day, CWSA, LAT, LONG, CUA)
polypara.dat	Input: initial S/E/I/R + rate parameters per polygon (8 rows) for the SPUR submodel
TIMENEW.DAT / TIMEOLD.DAT	Scratch/state files ping-ponged between SPUR timesteps
SUN.CSV	Output: header of user-entered parameters, then per-day Days, Umbra (SUSQI), Pores (SUSQS), CWSA,

Assessment for the Python port

- The interactive READ(*,*) prompts (umbra count, total spots, three percentages, and the Continue/Stop loop) map naturally to Python input() calls — or, better, function/CLI arguments so the model can run non-interactively or from a notebook cell.
- All of the EXPO/ENERGY/MASS/JTEMP/E2 math is straightforward scalar math (math.log, math.atan, math.exp, math.sqrt) — trivial to port one-to-one.
- SPUR's fixed-point relaxation loop and ran2's hand-rolled RNG are the fiddliest pieces to port faithfully. ran2 should probably become random.random() (or NumPy's Generator) unless bit-for-bit reproducibility of the original FORTRAN random sequence is required.
- The GOTO-based error handling and the LOC “line marker” pattern have no direct Python equivalent — ordinary try/except around file I/O replaces it more cleanly.
- Recommend replacing the file-based state ping-pong (TIMENEW.DAT/TIMEOLD.DAT) with an in-memory list of dicts/dataclasses per polygon, writing only SUN.CSV as the actual persistent output.

Open questions before porting: confirm the intended behavior of the underused K / ZCELLS branch (should the SEIR submodel actually evolve across calls, or is the current re-read-every-time behavior acceptable?), and whether SUN.CSV's exact column format needs to match the legacy output byte-for-byte.